

NumPy & Linear Algebra

Sattam Altwaim

sattam.tuuaim@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology



موهبة
Mawhiba

KAUST Academy & Mawhiba
AI Olympiad Training (Beginners)
Week 1, Day 05, Self-study

23 September 2026

1. Where This Fits
2. Learning Outcomes
3. Axis
4. Broadcasting
5. Boolean Masks
6. Reshape and Images
7. The Model Without Loops
8. Olympiad Practice
9. Check Yourself
10. Limitations and What Comes Next
11. Summary
12. References

- ▶ **In the lecture (Day 02):** a NumPy **array** is a vector or matrix stored as one object with a `shape`. You created arrays, indexed them (`A[1, 2]`, `A[:, 2]`), added and scaled them, used `@` for the dot and matrix products, `.T`, and `.mean()` on one column at a time with `X[:, j]`.
- ▶ **This deck adds** exactly what Day 02 marked “skipped on purpose”: **axis** (all columns at once), **broadcasting** (a matrix minus a vector), **boolean masks** (the rows of one species), **reshape** and an image as an array, and the nearest-centroid model of Day 02 and Day 04 rewritten without loops.
- ▶ **How to use it:** read a frame, type its code into a Colab cell, run it, compare with the printed output. The Iris arrays `X` (150 rows, 4 columns) and `y` (150 names) are loaded as in Day 02’s Lab 2. Answer the three questions at the end.

Where the lecture's version breaks

Day 02's Lab 2 needed a loop for every “per column” or “per species” question:

```
for j in range(4):                                # one column at a time
    print(X[:, j].mean())
sums = {name: np.zeros(4) for name in species_names}
for row, label in zip(X, y):                      # one row at a time
    sums[label] = sums[label] + row
```

Ten lines, two loops, and a dictionary, for two means. Week 3 trains on 60,000 images of 28×28 pixels: loops like these would take minutes per step.

After this deck you can

Write those two computations as `X.mean(axis=0)` and `X[y == name].mean(axis=0)`, read any array's shape and know what an operation will do to it, and turn an image into an array and back.

By the end of this deck, you will be able to:

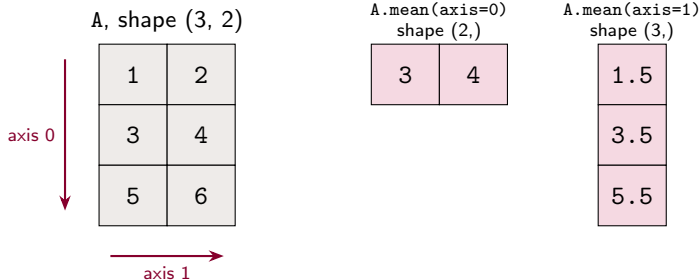
- ▶ **Explain** what `axis=0` and `axis=1` mean and predict the shape of `X.mean(axis=...)`.
- ▶ **Apply** broadcasting to subtract a vector from every row of a matrix, and read the error when shapes do not fit.
- ▶ **Select** rows with a boolean mask and combine conditions with `&` and `|`.
- ▶ **Reshape** an array, and explain how an image is a matrix of brightness values.
- ▶ **Rewrite** the nearest-centroid model with no Python loops, and check it gives the same accuracy.
- ▶ **Recognise** these operations in scikit-learn and PyTorch code (Weeks 2 to 4).

NumPy & Linear Algebra: **Axis**

Definition

A matrix has two **axes**: axis 0 runs **down the rows**, axis 1 runs **across the columns**.

`A.mean(axis=0)` collapses axis 0: it averages down each column and leaves one number per column. `A.mean(axis=1)` collapses axis 1: one number per row.



Rule of thumb: the axis you name is the one that **disappears** from the shape. (3, 2) with `axis=0` gives (2,); with `axis=1` gives (3,); with no axis, one number.

Column j of $\mathbf{A}$ has entries $a_{1j}, a_{2j}, \dots, a_{mj}$. Its mean is the Day 02 mean applied down that column:

$$\bar{a}_{.j} = \frac{1}{m} \sum_{i=1}^m a_{ij} \quad \mathbf{A} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}$$

- ▶ $m = 3$ rows. Column 1: $\bar{a}_{.1} = (1 + 3 + 5)/3 = 3$. Column 2: $(2 + 4 + 6)/3 = 4$. So `A.mean(axis=0)` is $(3, 4)$.
- ▶ Row means, `axis=1`: $(1 + 2)/2 = 1.5$, $(3 + 4)/2 = 3.5$, $(5 + 6)/2 = 5.5$.
- ▶ The same rule for `sum`, `std`, `min`, `max`: `A.sum(axis=0)` is $(9, 12)$, `A.sum()` with no axis is 21.
- ▶ `A.argmax(axis=0)` gives the *row index* of the largest entry in each column: $(2, 2)$, because both columns peak in row 2.

```
A = np.array([[1, 2], [3, 4], [5, 6]])
A.mean(axis=0)          # [3. 4.]           per column
A.mean(axis=1)          # [1.5 3.5 5.5]      per row
A.sum(axis=0)           # [ 9 12]
A.argmax(axis=0)        # [2 2]           row index of each column's max

X.shape                 # (150, 4)           Iris, from Day 02's Lab 2
X.mean(axis=0)          # [5.843 3.054 3.759 1.199]
X.std(axis=0)           # [0.825 0.432 1.759 0.761]
X.max(axis=0)           # [7.9 4.4 6.9 2.5]
```

- ▶ The Day 02 loop over `X[:, j]` is now one line. `pandas' describe()` (Day 03) calls exactly these.
- ▶ Where you will meet it: an image batch in Week 4 has shape (64, 28, 28); `batch.mean(axis=0)` is the average image. Same rule, one more axis.

NumPy & Linear Algebra: **Broadcasting**

Definition

When two arrays of different shapes meet in $+$, $-$, $*$ or $/$, NumPy **broadcasts** the smaller one: it is reused along the missing axis, as if copied for every row, without making the copies.

$$\mathbf{A} - \mathbf{m} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix} - \begin{bmatrix} 3 & 4 \end{bmatrix} = \begin{bmatrix} 1-3 & 2-4 \\ 3-3 & 4-4 \\ 5-3 & 6-4 \end{bmatrix} = \begin{bmatrix} -2 & -2 \\ 0 & 0 \\ 2 & 2 \end{bmatrix}$$

- ▶ $\mathbf{m} = (3, 4)$ is the column means from the last section. Subtracting it from every row **centres** the data: each column now has mean 0.
- ▶ **The rule:** compare the shapes from the right. $(3, 2)$ and $(2,)$: the last numbers match (2 and 2), the missing axis is filled by repeating. Two shapes fit when, from the right, each pair of numbers is equal or one of them is 1.
- ▶ A single number broadcasts to everything: $\mathbf{A} * 10$ multiplies all six entries, as in Day 02.

```
A - A.mean(axis=0)          # [[-2 -2] [0 0] [2 2]]   (3,2)-(2,)
A / np.array([1, 2])        # [[1 1] [3 2] [5 3]]   per column

Z = (X - X.mean(axis=0)) / X.std(axis=0)    # centre, then scale
Z.mean(axis=0)          # [0. 0. 0. 0.]   (up to rounding)
Z.std(axis=0)           # [1. 1. 1. 1.]
Z[0]                    # [-0.901  1.032 -1.341 -1.313]
```

- ▶ Row 0 of Z says: this flower's sepal length is 0.9 standard deviations *below* average, its petal length 1.3 below. Every column now uses the same unit, “standard deviations from the mean” (Day 02's σ).
- ▶ **Where you will meet it:** this is **standardisation**, the first step of most Week 2 models (StandardScaler does these two lines). Day 02's Lab 2 reflection asked why the sepal columns hurt the distance; scaling is the answer.

```
A.shape          # (3, 2)
A - np.array([1, 2, 3]) # ValueError: operands could not be
                        # broadcast together with shapes (3,2) (3,)
```

Read the shapes in the message

From the right: 2 versus 3. Neither is 1 and they differ, so the rule fails. The vector has one number per *row*, but broadcasting lines it up with the *columns*.

- ▶ **The fix:** give the vector a column shape, `v.reshape(3, 1)`, so the shapes read (3,2) and (3,1): 2 versus 1 fits, 3 versus 3 fits. Reshape is the next section but one.
- ▶ **The silent version:** (3,3) minus (3,) works and subtracts along the columns even if you meant rows. No error, wrong numbers. Check `.shape` before and after.

NumPy & Linear Algebra: **Boolean Masks**

```
y[:3]                # ['Iris-setosa' 'Iris-setosa' 'Iris-setosa']
mask = (y == "Iris-setosa")
mask[:3]             # [ True  True  True]  one per row
mask.shape           # (150,)
mask.sum()           # 50          True counts as 1
(X[:, 2] > 5).sum()   # 42          petal length above 5 cm
```

Definition

A **boolean mask** is an array of True/False with one entry per row (or per element). Comparing an array with a value asks the question for every entry at once. Summing a mask counts the Trues.

This is Day 01's "count with an if inside a loop" and Day 03's `df[df["Age"] < 18]`, at array speed. pandas filters are masks underneath.

```
setosa = X[y == "Iris-setosa"]      # keep rows where mask is True
setosa.shape                        # (50, 4)
setosa.mean(axis=0)                 # [5.006  3.418  1.464  0.244]

both = (y == "Iris-setosa") & (X[:, 2] < 1.2)  # & is AND, | is OR
both.sum()                                # 2
X[both]                                  # shape (2, 4): the two rows
```

- ▶ `X[mask]` keeps exactly the rows whose mask entry is True: 50 of 150 here. Then `axis=0` averages them.
- ▶ Day 02's ten-line centroid loop is now `X[y == name].mean(axis=0)`.
- ▶ Each condition goes in its own brackets before `&` or `|`, exactly as in pandas (Day 03). `and/or` do not work on arrays.

```
img = digits.images[0].copy()      # 8x8 image, values 0 to 16
img[img < 5] = 0                    # erase faint pixels
(img > 0).sum()                      # 28 pixels still on

scores = np.array([78, 92, 85, 64, 91])
scores[scores < 70] = 70            # [78 92 85 70 91]
```

- ▶ `arr[mask] = value` changes only the True positions, in place. It is the array version of Day 03's `fillna` and of “clip outliers”.
- ▶ **Where you will meet it:** `y_pred[probabilities > 0.5] = 1` turns Week 2's probabilities into labels; a segmentation mask in Week 4 is literally a boolean image.

NumPy & Linear Algebra: **Reshape and Images**

```
v = np.arange(6)           # [0 1 2 3 4 5]          shape (6,)
v.reshape(2, 3)            # [[0 1 2] [3 4 5]]      shape (2, 3)
v.reshape(3, -1)          # shape (3, 2): -1 = "work it out"
v.reshape(2, 3).reshape(-1) # [0 1 2 3 4 5]      again: flatten
v.reshape(4, 2)           # ValueError: cannot reshape array of size 6
                          #    into shape (4,2)
```

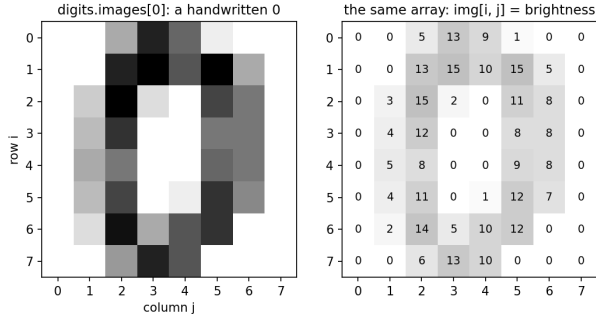
Definition

`reshape` keeps the numbers and their order and changes only the shape. The product of the new shape must equal the number of elements: $6 = 2 \times 3 = 3 \times 2$, but $4 \times 2 = 8 \neq 6$.

Numbers are read row by row: the first row of `v.reshape(2, 3)` is the first three numbers.

Used constantly to hand data to models: a matrix of images flattened to rows (next frame), or a vector turned into a column with `reshape(-1, 1)` so it broadcasts down rows.

An Image Is a Matrix



Source: `scripts/day05_numpy_figures.py`, `scikit-learn load_digits` (1,797 handwritten digits, 8×8 pixels).

- ▶ A grey image is a matrix of brightness values: here 0 is white and 16 is black. `img[i, j]` is the pixel in row i , column j ; `img[0]` is the top row.
- ▶ Everything from this deck applies: `img.mean()` is the average brightness (4.59), `img > 8` masks the dark strokes, `img.argmax(axis=1)` finds the darkest column of each row.

```
from sklearn.datasets import load_digits
digits = load_digits()
digits.images.shape          # (1797, 8, 8)   1797 images of 8x8
digits.data.shape            # (1797, 64)     one row per image
digits.images[0].reshape(-1) # 64 numbers: image as a vector
digits.data[0].reshape(8, 8) # back to a picture
(digits.images[0].reshape(-1) == digits.data[0]).all() # True
```

- ▶ A dataset of images is a 3-axis array: image, row, column. Flattening each image to 64 numbers gives the 1797×64 matrix that Week 2's models expect: one row per example, as in Iris (150×4).
- ▶ Week 4's convolutional networks work on the 8×8 (or 28×28) shape directly; reshape moves between the two views for free.
- ▶ **Skipped on purpose:** colour images add a fourth axis (red, green, blue). Week 4, Day 22.

NumPy & Linear Algebra: **The Model Without Loops**

The three centroids stacked as a 3×4 matrix **C**, one flower **x** (row 100, a virginica):

```
names = ["Iris-setosa", "Iris-versicolor", "Iris-virginica"]
C = np.array([X[y == n].mean(axis=0) for n in names]) # (3, 4)
x = X[100] # [6.3 3.3 6. 2.5]
C - x # (3,4) minus (4,): broadcast
dists = np.linalg.norm(C - x, axis=1) # [5.23 2.195 0.784]
dists.argmax() # 2
names[dists.argmax()] # 'Iris-virginica'
```

- ▶ Step by step: broadcasting subtracts **x** from every centroid; axis=1 takes the Day 02 norm of each difference row; `argmin` gives the row index of the smallest, which is the species.
- ▶ Day 04's predict had a loop over the centroids with `best` and `best_dist`. These three lines replace it.

```
D = np.linalg.norm(X[:, None, :] - C[None, :, :], axis=2) # (150, 3)
D[100] # [5.23 2.195 0.784]
pred = np.array(names)[D.argmin(axis=1)] # one name per flower
(pred == y).mean() # 0.9266... no loop
```

- ▶ $X[:, \text{None}, :]$ has shape (150,1,4) and $C[\text{None}, :, :]$ has (1,3,4). From the right: 4 and 4 match, 1 versus 3 and 150 versus 1 broadcast. The difference is (150,3,4): every flower minus every centroid.
- ▶ $\text{axis}=2$ collapses the 4 measurements into one norm, leaving (150,3): a distance table, flowers by species. $\text{argmin}(\text{axis}=1)$ picks the closest species per row.
- ▶ Same 0.927 as the loops of Day 02 and Day 04. This is what “vectorised” means, and why NumPy was 25 to 50 times faster on Day 01’s chart.
- ▶ **Skipped on purpose:** `None` as a new axis is the one advanced trick here; you may keep the one-flower version and a `for` loop over rows. Both are correct.

NumPy & Linear Algebra: **Olympiad Practice**

Task

Write `reshape_matrix(a, new_shape)` where `a` is a matrix given as a list of rows and `new_shape` is a pair (`rows`, `cols`). Return the reshaped matrix as a list of rows, reading the numbers row by row. If the new shape does not hold the same number of elements, return `[]`.

- ▶ Example: `a = [[1, 2, 3, 4], [5, 6, 7, 8]]`, `new_shape = (4, 2)` $\rightarrow$ `[[1, 2], [3, 4], [5, 6], [7, 8]]`.
- ▶ Example: same `a`, `new_shape = (3, 3)` $\rightarrow$ `[]` (8 elements cannot fill 9 cells).
- ▶ Scoring: hidden test cases. NumPy is allowed, but the answer must be a plain list of lists.

Adapted from: Deep-ML Open Problem Bank, problem 3 "Reshape Matrix" (easy).

Recognise: the reshape rule (element count must match) plus a conversion back to lists.

Check: the two examples above, and a 1×6 to 6×1 case.

```
import numpy as np

def reshape_matrix(a, new_shape):
    arr = np.array(a)
    rows, cols = new_shape
    if arr.size != rows * cols:          # the reshape rule
        return []
    return arr.reshape(rows, cols).tolist()  # list of rows

a = [[1, 2, 3, 4], [5, 6, 7, 8]]
reshape_matrix(a, (4, 2))    # [[1, 2], [3, 4], [5, 6], [7, 8]]
reshape_matrix(a, (3, 3))    # []
```

Without NumPy: flatten with a double loop into one list, check its length, then slice it into rows of cols items. Same idea, twelve lines.

NumPy & Linear Algebra: **Check Yourself**

Answer on paper before turning the page. B is the matrix below.

```
B = np.array([[2, 4, 6],  
             [1, 3, 5]])
```

1. What are `B.mean(axis=0)`, `B.mean(axis=1)` and their shapes?
2. `B - np.array([1, 1, 1])` works; `B - np.array([1, 1])` raises an error. Explain both with the broadcasting rule, and give a one-line fix that subtracts 1 from the first row and 1 from the second using a `(2, 1)` array.
3. What does `B[B > 3].reshape(-1)` contain, and how many elements does `B.reshape(3, 2)` have per row?

1. `axis=0` averages down the columns: `(1.5, 3.5, 5.5)`, shape `(3, )`. `axis=1` averages across each row: `(4, 3)`, shape `(2, )`. The named axis disappears.
2. Shapes from the right: `(2, 3)` and `(3, )`: 3 matches 3, the missing axis repeats the vector for both rows, so it works. `(2, 3)` and `(2, )`: 3 versus 2, neither is 1, so it fails. Fix: `B = np.array([1, 1]).reshape(2, 1)`, shapes `(2, 3)` and `(2, 1)`: 3 versus 1 broadcasts, 2 matches 2.
3. `B > 3` is `[[False, True, True], [False, False, True]]`; `B[B > 3]` keeps the True entries in row order: `[4, 6, 5]`, already flat. `B.reshape(3, 2)` has 6 elements in 3 rows of 2: `[[2, 4], [6, 1], [3, 5]]`.

NumPy & Linear Algebra: **Limitations & What Comes Next**

The mistakes to watch for

- ▶ **Wrong axis, no error:** `X.mean(axis=1)` on Iris gives 150 numbers (one per flower) when you wanted 4. Check the shape of the result against what you expected.
- ▶ **Silent broadcasting:** a square matrix minus a vector subtracts along the columns even when you meant rows. Reshape the vector to $(n, 1)$ to go down rows.
- ▶ **and instead of &:** `ValueError: The truth value of an array ... is ambiguous.` Masks combine with `&`, `|`, `~`, each condition in brackets.
- ▶ **Reshape that scrambles:** `reshape` never moves numbers, it re-cuts the sequence. To swap rows and columns you want `.T`, not `reshape`.
- ▶ **Skipped on purpose:** views versus copies, fancy indexing with index arrays, `np.where`, sorting, and random numbers. They appear where a lab needs them.
- ▶ **Tomorrow, Day 06 (Olympiad practice):** pandas cleaning and EDA tricks for Friday's exercise; every `groupby` and `filter` there is an axis or a mask underneath.
- ▶ **Week 3:** PyTorch tensors have the same shape, axis (called `dim`), broadcasting and `reshape` (called `view`). Nothing here has to be relearned.

NumPy & Linear Algebra: **Summary**

- ▶ **Axis:** `axis=0` collapses down the rows (one number per column), `axis=1` across the columns; the named axis disappears from the shape.
- ▶ **Broadcasting** reuses a smaller array along the missing axis; shapes fit when, from the right, each pair is equal or one is 1. Centre and scale is $(X - X.\text{mean}(\text{axis}=0)) / X.\text{std}(\text{axis}=0)$.
- ▶ A **boolean mask** answers a question for every row at once; `X[mask]` keeps rows, `mask.sum()` counts, `arr[mask] = v` writes; combine with `&` and `|`.
- ▶ **Reshape** re-cuts the same numbers into a new shape with the same element count; an **image** is a matrix of brightness values, and a dataset of images flattens to one row per image.
- ▶ The nearest-centroid **model without loops**: `np.linalg.norm(C - x, axis=1).argmin()`, same 0.927 accuracy, at array speed.
- ▶ Every wrong-shape mistake is caught by printing `.shape` before and after.

NumPy & Linear Algebra: **References & Resources**

- [1] Harris, C. R. et al. (2020). Array programming with NumPy. *Nature* 585, 357–362.
Broadcasting rules:
<https://numpy.org/doc/stable/user/basics.broadcasting.html>
- [2] Open-Deep-ML (2025). *DML-OpenProblem*, problem 3: Reshape Matrix. The practice task is adapted from it. <https://github.com/Open-Deep-ML/DML-OpenProblem>
- [3] Alpaydin, E. and Kaynak, C. (1998). *Optical Recognition of Handwritten Digits* data set, via scikit-learn's `load_digits`; used for the image figure. https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_digits.html
- [4] Fisher, R. A. (1936). *Iris* data set, Kaggle copy `uciml/iris`; all Iris numbers in this deck. <https://www.kaggle.com/datasets/uciml/iris>

- [5] Altwaim, S. (2026). *Day 1: Math & Code Foundations*, KAUST Academy & Mawhiba IOAI 2026 training. The broadcasting and indexing frames are adapted from its NumPy section.